00 01 02 **03** 04 05 06 07 08 09 0A 0B 0C 0D

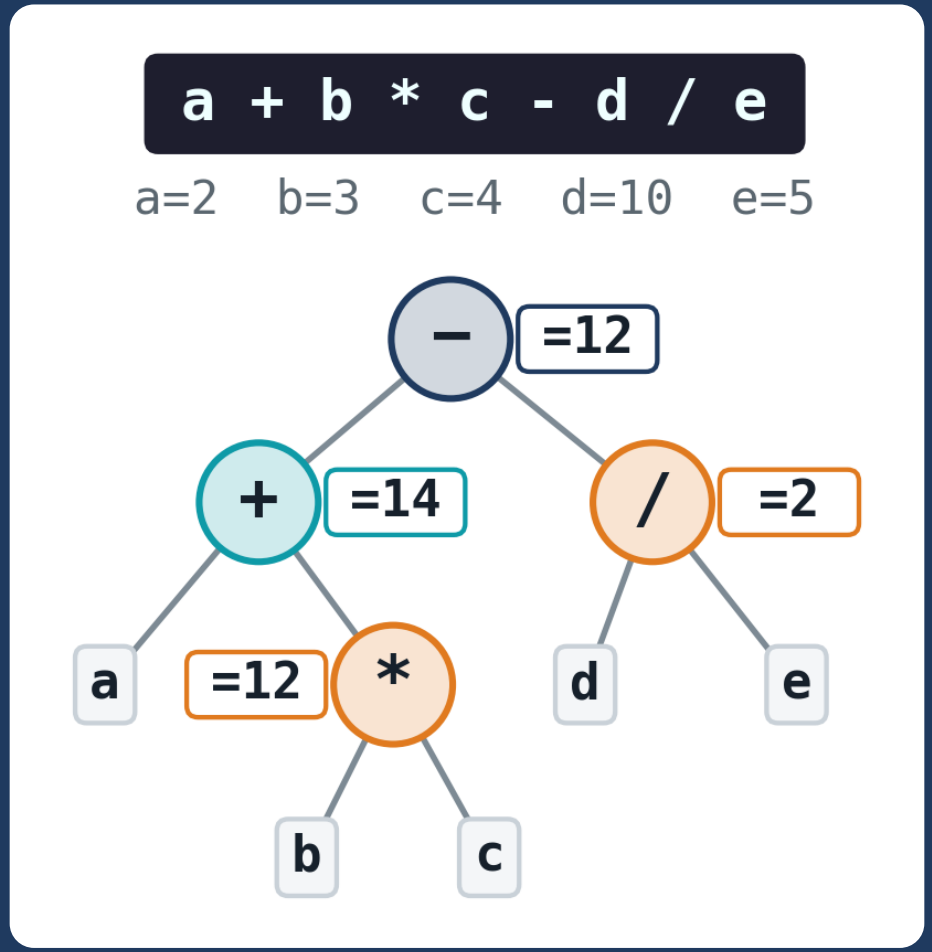
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 04

Operatörler, İfadeler ve Tip Dönüşümleri: ALU’nun Dili

İşlemci veriyi nasıl hesaplar, C bir ifadeyi hangi sırayla çözer, tip değiştirirken bitlere ne olur?

MODÜL II · VERİNİN TEMSİLİ VE İŞLENMESİ



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Bilgisayar ve Program	**H2** Algoritmik Düşünme	**H3** Değişkenler ve Veri Tipleri	**H4** **Operatörler ve İfadeler**	**H5** Karar Yapıları	**H6** Döngüler I	**H7** Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Fonksiyonlar I	**H9** Kapsam, Yığın, Özyineleme	**H10** Diziler I	**H11** Diziler II: Matrisler	**H12** İşaretçiler ve Bellek	**H13** Karakter Dizileri	**H14** Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

Geçen haftadan: üç satır, üç tahmin

H04_K12_geri_cagirma.c · satır 11–17

```
11      unsigned int u = UINT_MAX;
12      u = u + 1;
13      printf("%u\n", u);
14
15      printf("%d\n", 0.1 + 0.2 == 0.3);
16
17      printf("[%6.2f]\n", 3.14159);
```

S1 `UINT_MAX + 1` ne yazar? Tanımlı mı? `INT_MAX + 1` neden farklı?

S2 `0.1 + 0.2 == 0.3` neden **0**? Kesirli sayılar nasıl karşılaştırılır?

S3 `%6.2f` ile 3.14159 nasıl yazılır?

Önce kâğıda tahmin edin; eşinizle karşılaştırın. Yanıtlar bir sonraki slaytta, **gerçek çıktıyla**.

Geçen haftadan · yanıtlar (gerçek çıktı: H04_K12_geri_cagirma.c)

Gerçek çıktı satırları: `0 · 0 · [  3.14]`

S1 · 0

İşaretsiz aritmetik **2^{32} modunda tanımlıdır**: `UINT_MAX + 1` → 0. İşaretli `INT_MAX + 1` ise **tanımsız davranıştır**.

S2 · 0

0.1 + 0.2 saklanan 0.3'ün **komşu double'ına** düşer. Kesirli sayıları farkla karşılaştırın: `fabs(a - b) < 1e-9`.

S3 · [3.14]

Alan 6, ondalık 2; sağa yaslı. Bu hafta ortalamaları **yuvarlarken** `%.0f`'in bir sürprizini göreceğiz.

Haftanın Büyük Sorusu

Tek bir tip dönüşümü, yüz milyonlarca dolarlık bir roketi nasıl havada parçalayabilir?



Bu hafta ifadeleri, operatörleri ve **tip dönüşümlerini** öğreneceğiz. Haftanın sonunda aynı dönüşümü kendi makinemizde deneyeceğiz: C'de sonuç derleyiciye göre **−25536**, **32767** ya da anlamsız bir sayı çıkacak (Hata Avcısı 7).

Kaynak: Lions (1996), ARIANE 5 Flight 501 Failure: Report by the Inquiry Board.

GERÇEK DÜNYADAN

1996 · Uçuş 501

4 Haziran 1996, Kourou: Ariane 5’in 501 sayılı uçuşu

- Uçuş dizisi başladıktan (H0: ana motor ateşleme dizisi) **yaklaşık 40 saniye** sonra, **≈ 3700 m** irtifada roket rotasından saptı, parçalandı ve patladı.
- **H0 + 36,7 s** (kalkıştan ≈ 30 s sonra): eylemsizlik referans sistemlerinde (SRI) 64 bitlik **BH** (yatay hızla ilişkili hizalama değeri) 16 bitlik işaretli tamsayıya sığmadı → **işlenen hatası** (Operand Error).
- Dönüşüm **korumasızdı**; yazılım **Ariane 4’ten yeniden kullanılmıştı**, Ariane 5’in yatay hızları çok daha büyüktü.
- Yedek ve etkin SRI **aynı yazılımla** durdu; ana bilgisayar tanı verisini uçuş verisi sandı, nozulları aşırı saptırdı.

Lions, J. L. (1996). ARIANE 5 flight 501 failure: Report by the inquiry board. ESA.

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 4.1	Aritmetik, atama, artırma/azaltma, ilişkisel ve mantıksal operatörleri doğru kullanır.
ÖÇ 4.2	Operatör önceliği ve birleşme yönünü uygulayarak ifadelerin değerini elle hesaplar; ifade ağacı çizer.
ÖÇ 4.3	Örtük (implicit) ve açık (explicit, cast) tip dönüşümlerinin sonuçlarını öngörür.
ÖÇ 4.4	Bit düzeyi operatörlerle maskeleme, kaydırma ve bayrak işlemleri yapar.
ÖÇ 4.5	math.h kütüphane fonksiyonlarını kullanarak matematiksel formülleri koda aktarır.

01

Aritmetik, atama ve artırma

İfade · ALU · tamsayı bölmesi · kalan · bileşik atama · i++ / ++i

KAVRAM

İfade = işlenenler + operatörler



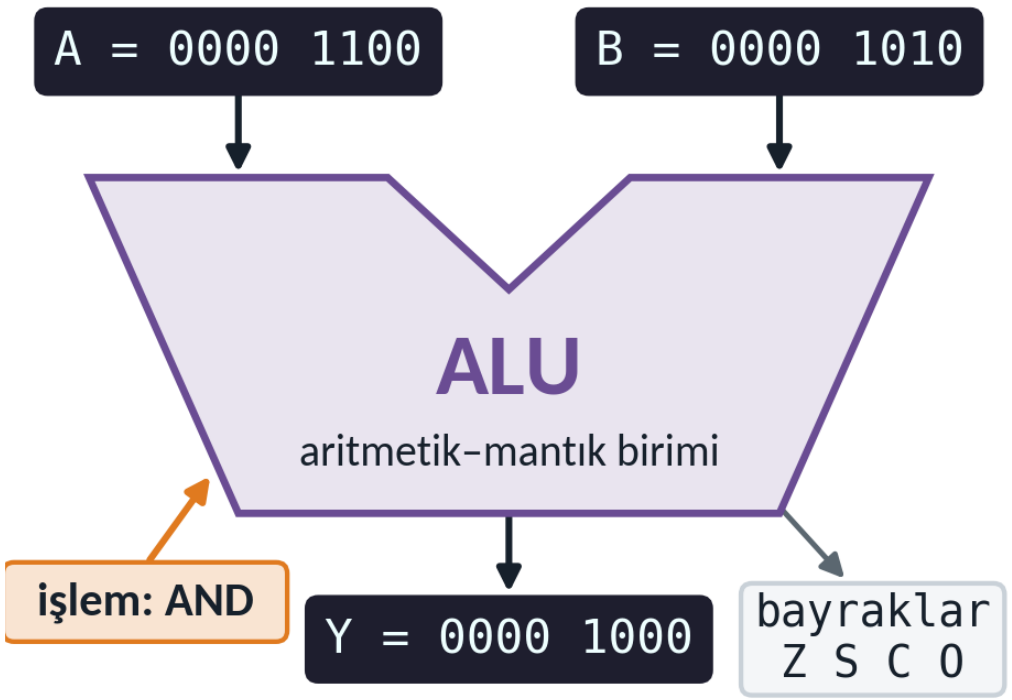
Tekli (unary)
`-x !x ~x x++`

İkili (binary)
`a + b a < b a & b`

Üçlü (ternary)
`k ? a : b` (H5)

Her ifadenin bir **tipi** ve **değeri** vardır: `7 / 2` → `int`, 3. Bazılarının ayrıca **yan etkisi** vardır: `x = 5`, `i++`, `printf(…)`.

Kaputun altında: her operatör bir ALU komutu




```
12 & 10 = 8   →   tek komut: and
```

C operatörü	x86-64 komutu	birim
`+  -`	`add  sub`	ALU
`&  |  ^`	`and  or  xor`	ALU
`~`	`not`	ALU
`<<  >>`	`shl  shr/sar`	ALU (kaydırıcı)
`<  ==  >=`	`cmp + bayrak`	ALU
`*`	`imul`	çarpma birimi
`/  %`	`idiv / div`	bölme birimi

- ALU iki sayıyı alır, **seçilen işlemi** yapar; sonucu ve **bayrakları** (Z sıfır, S işaret, C elde, O taşma) üretir. Karşılaştırmalar `cmp` ile bayrakları ayarlar (H5: koşullu atlama).
- Derleyici her operatörü bir ya da birkaç komuta çevirir; çarpma ve özellikle **bölme** ayrı, daha yavaş devrelerdedir.

Tamsayı bölmesi sıfıra doğru keser; % bölünenin işaretini alır

C99'dan beri tamsayı bölmesi SIFIRA DOĞRU keser (kesirli kısım atılır)



Gerçek çıktı (H04_K15_bolme_isaretleri.c)

```
 7 /  2 =  3     7 %  2 =  1
-7 /  2 = -3    -7 %  2 = -1
 7 / -2 = -3     7 % -2 =  1
-7 / -2 =  3    -7 % -2 = -1
```

Her zaman geçerli özdeşlik

```
(a / b) * b + a % b == a

(-3) * 2 + (-1) = -7   ✓
```

Kalanın işareti = bölünenin işareti · b == 0: tanımsız

- İki `int` → **tamsayı bölmesi**: `7 / 2` = 3.
- C99'dan beri **sıfıra doğru**: `-7 / 2` = **–3** (–4 değil).
- Özdeşlik: `(a / b) * b + a % b == a`.
- Bu yüzden `-7 % 2` = **–1**: C'nin `%`'i bir **kalan** operatörüdür.
- `%` yalnızca tamsayılarla; kesirli için `fmod`.

Örnek 1 ★ Dört işlem ve kalan

H04_01_dort_islem.c · satır 15–21

```
15      printf("a + b = %d\n", a + b);
16      printf("a - b = %d\n", a - b);
17      printf("a * b = %d\n", a * b);
18      printf("a / b = %d   (tamsayı bölmesi: kesir atılır)\n", a / b);
19      printf("a %% b = %d   (kalan)\n", a % b);
20      printf("(a / b) * b + a %% b = %d   (a ile aynı olmalı)\n",
21             (a / b) * b + a % b);
```

ÇIKTI

```
$ ./dort_islem
İki tamsayı girin (a b): -7 2
a + b = -5
a - b = -9
a * b = -14
a / b = -3   (tamsayı bölmesi: kesir atılır)
a % b = -1   (kalan)
(a / b) * b + a % b = -7   (a ile aynı olmalı)
```

Tamsayıda sıfıra bölme: tanımsız davranış

GERÇEK ÇIKTI · BU MAKİNEDEKİ GÖZLEM

```
$ ./dort_islem      # b = 0
İki tamsayı girin (a b): 17 0
a + b = 17
a - b = 17
a * b = 0
bash: line 1:  8627 Floating point exception./dort_islem
[çıkış kodu: 136]
```



C standardı: `a / 0` ve `a % 0` **tanımsızdır.**

- x86-64'te `idiv` komutu sıfır bölenle **donanım istisnası** üretir.
- Linux programı **SIGFPE** ile sonlandırır: çıkış kodu 136 = 128 + 8.
- "Floating point" adına aldanmayın: hata **tamsayı** bölmesinde.

Kayan noktalı bölmede farklı: IEEE 754 → `1.0 / 0.0` = **inf**, `0.0 / 0.0` = **nan** (H3, Örnek 10)

Önce denetle: bu hafta **kısa devreyle** `(b != 0) && (a / b > 1)`, H5'te `if (b != 0)`

Atama ve bileşik atama

H04_K11_bilesik_atama.c · satır 10–23

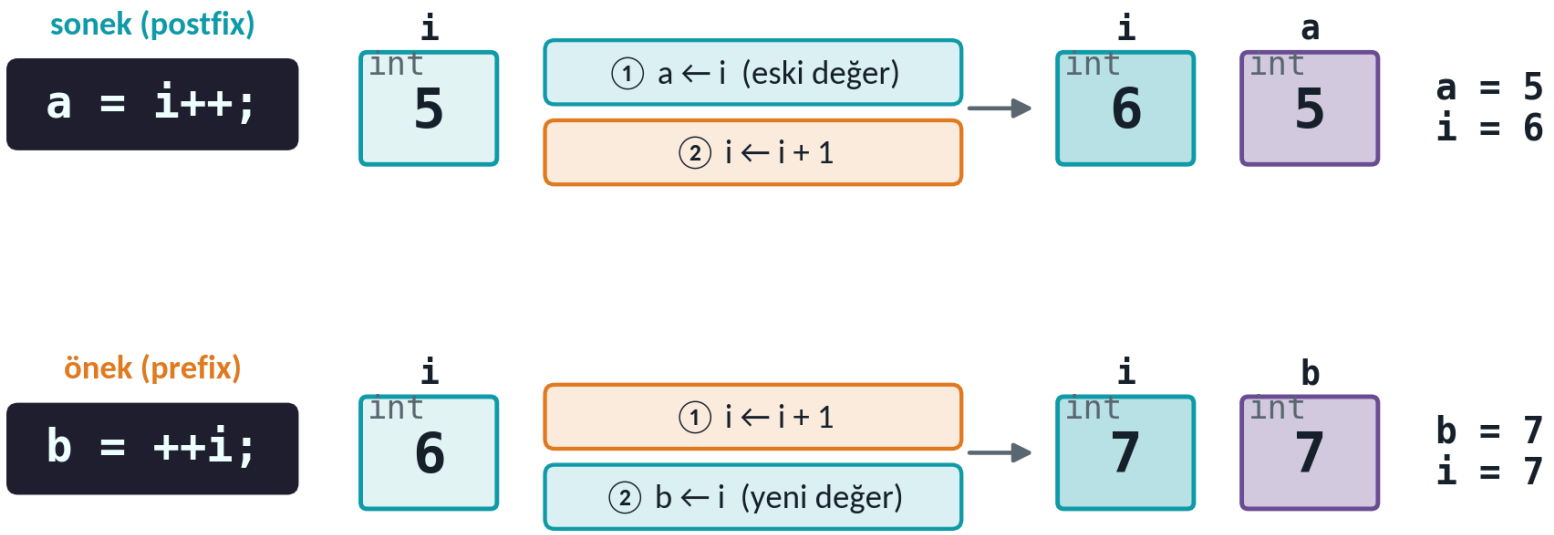
```c
    int x = 20;

    x += 5;          /* x = x + 5 */
    printf("x += 5      -> %d\n", x);
    x -= 3;
    printf("x -= 3      -> %d\n", x);
    x *= 2;
    printf("x *= 2      -> %d\n", x);
    x /= 5;          /* tamsayı bölmesi */
    printf("x /= 5      -> %d\n", x);
    x %= 3;
    printf("x %%= 3      -> %d\n", x);
    x *= 2 + 1;      /* x = x * (2 + 1): önce sağ taraf */
    printf("x *= 2 + 1 -> %d\n", x);
```

Açılımı	x
x = x + 5	25
x = x - 3	22
x = x * 2	44
x = x / 5	8
x = x % 3	2
x = x * (2 + 1)	6

= bir **operatördür**; değeri atanan değerdir; **sağdan sola**: x = y = z = 7.
x *= 2 + 1 → x * 3.

++ ve --: önek mi, sonek mi?



Tek başına bir deyim olarak (i++; ya da ++i;) ikisi arasında fark yoktur.

- **Sonek** `i++`: ifadenin değeri **eski** değer; artırma yan etki.
- **Önek** `++i`: önce artırır; değer **yeni** değer.
- Tek başına deyim olarak (`i++;`) fark **yok**.
- Aynı değişkeni bir ifadede **iki kez** değiştirmeyin (Bölüm 3).

TAHMİN ET

Örnek 5 · PRIMM: i = 7 iken c ve d ne olur?

H04_05_artirma_azaltma.c · satır 18–22

```
18      int c = i--;
19      printf("c = i--  ->  c = %d, i = %d\n", c, i);
20
21      int d = --i;
22      printf("d = --i  ->  d = %d, i = %d\n", d, i);
```

A c = 6, d = 5

B c = 7, d = 5

C c = 7, d = 6

D c = 6, d = 6

Önce kâğıda yazın, eşinizle karşılaştırın; sonra programı çalıştırıp gerçek çıktıyla doğrulayın.

02

Karşılaştırma, mantık ve bitler

0/1 doğruluk değeri · && || ! · kısa devre · & | ^ ~ << >> · maskeler

İlişkisel operatörler: sonuç her zaman 0 ya da 1

H04_K02_iliskisel_mantiksal.c · satır 12–16

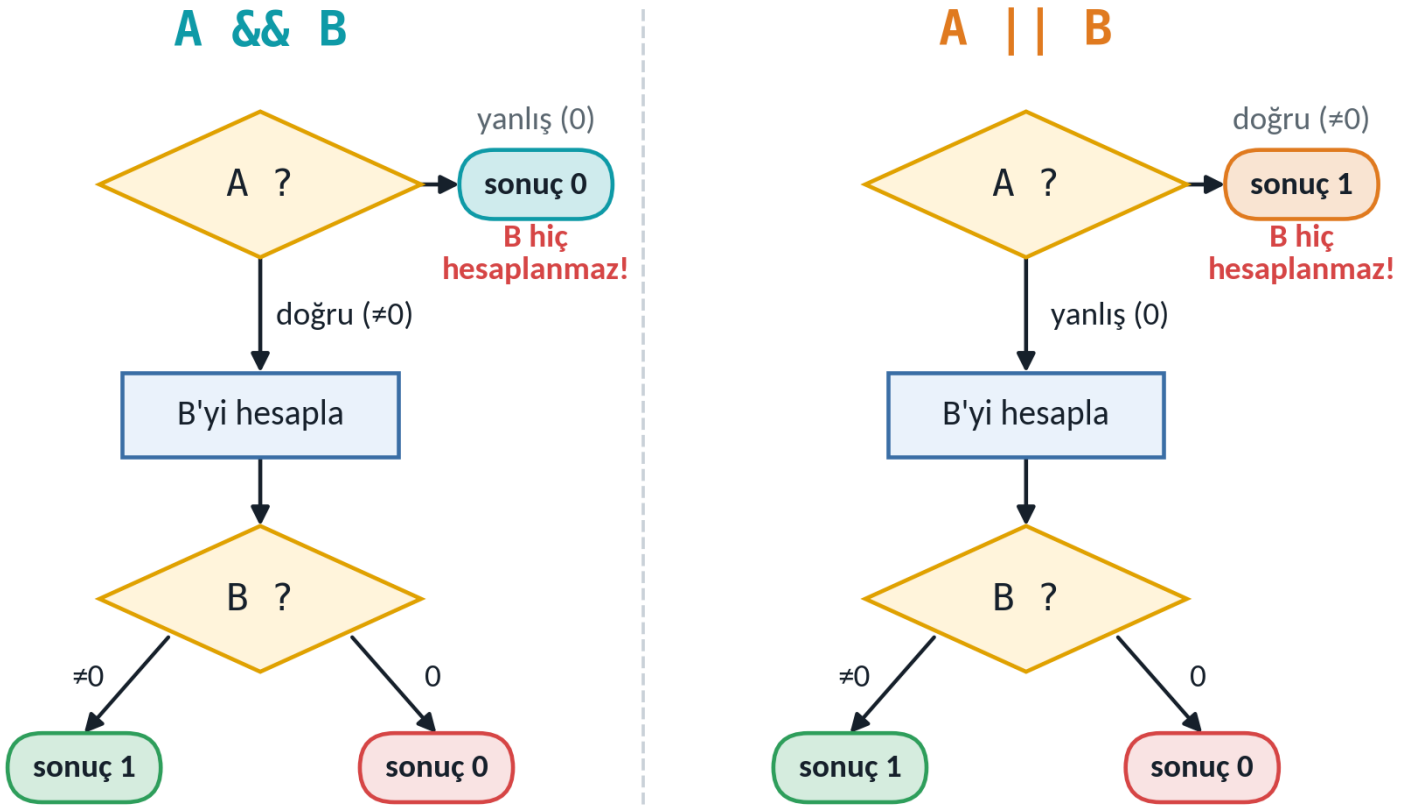
```
12      printf("x > 5  : %d\n", x > 5);
13      printf("x == 5 : %d\n", x == 5);
14      printf("x != 5 : %d\n", x != 5);
15      printf("!x     : %d\n", !x);
16      printf("!!x    : %d\n", !!x);
```

ÇIKTI (İLK 5 SATIR)

```
$ ./iliskisel
x > 5  : 1
x == 5 : 0
x != 5 : 1
!x     : 0
!!x    : 1
```

- `< <= > >= == !=` → **int** 0/1.
- Koşulda: **0 yanlış**, sıfırdan farklı her şey **doğru**.
- `!!x`: “x sıfırdan farklı mı?” (0/1).
- 0/1 sayıdır: `28 + artik`, `(x > 5) + (x > 6)`.

&& ve ||: soldan sağa ve kısa devreli



- `A && B`: A **yanlışsa** sonuç 0; **B hesaplanmaz**.
- `A || B`: A **doğruysa** sonuç 1; **B hesaplanmaz**.
- `&&`, `||`'den **sıkı bağlar**: `a || b && c` = `a || (b && c)`.
- Koruyucu kalıp: `(b != 0) && (a / b > 1)`.

Kısa devreyi görmek: sağ tarafa bir printf koyalım

H04_K03_kisa_devre.c · satır 13–25

```
13      int s1 = (x != 0) && printf("   sağ taraf çalıştı\n");
14      printf("   sonuç = %d\n", s1);
15
16      printf("2) (x == 0) && printf(...)\n");
17      int s2 = (x == 0) && printf("   sağ taraf çalıştı\n");
18      printf("   sonuç = %d\n", s2);
19
20      printf("3) (x == 0) || printf(...)\n");
21      int s3 = (x == 0) || printf("   sağ taraf çalıştı\n");
22      printf("   sonuç = %d\n", s3);
23
24      int b = 0;
25      int guvenli = (b != 0) && (10 / b > 1);   /* 10 / b hiç hesaplanmaz */
```

GERÇEK ÇIKTI ($ ./KİSA_DEVRE) · 1–2

```
1) (x != 0) && printf(...)
   sonuç = 0
2) (x == 0) && printf(...)
   sağ taraf çalıştı
   sonuç = 1
```

GERÇEK ÇIKTI · 3–4

```
3) (x == 0) || printf(...)
   sonuç = 1
4) (b != 0) && (10 / b > 1) = 0
```

Bit düzeyi operatörler: her bit komşusundan bağımsız

& (VE)

a	b	sonuç
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

| (VEYA)

a	b	sonuç
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**1**

^ (ÖZEL VEYA)

a	b	sonuç
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

~ (DEĞİL)

a	~a
0	**1**
1	**0**

Her bit komşusundan bağımsız işlenir.

```
a          1 1 0 0 1 0 1 0   0xCA = 202
b          0 1 0 1 1 1 0 0   0x5C = 92
-------------------------------------
a & b      0 1 0 0 1 0 0 0   0x48 = 72
a | b      1 1 0 1 1 1 1 0   0xDE = 222
a ^ b      1 0 0 1 0 1 1 0   0x96 = 150
~a & 0xFF  0 0 1 1 0 1 0 1   0x35 = 53
```

Örnek değerler H04_K04_bitsel.c programının gerçek çıktısından alınmıştır.

- & VE, | VEYA, ^ ÖZEL VEYA, ~ DEĞİL.
- && iki doğruluk değerinden 0/1 üretir; & **32 bitin her birini** işler.
- ~a 32 bitin tamamını çevirir → alt 8 bit için & 0xFF.
- Bit örneklerinde **işaretsiz** tipler kullanın.

Maskeler ve bayraklar: set, clear, toggle, test

SET · biti 1 yap

```
x |= 1u << 2;
```

	7	6	5	4	3	2	1	0
x	1	0	1	0	0	0	1	0
m	0	0	0	0	0	1	0	0
sonuç	1	0	1	0	0	1	1	0

Diğer bitler aynen kalır; bit 2 artık 1.

`x = 0xA2, m = 1u << 2 = 0x04`

CLEAR · biti 0 yap

```
x &= ~(1u << 5);
```

	7	6	5	4	3	2	1	0
x	1	0	1	0	0	0	1	0
~m	1	1	0	1	1	1	1	1
sonuç	1	0	0	0	0	0	1	0

~m yalnızca bit 5'te 0: o bit silinir.

`x = 0xA2, m = 1u << 5 = 0x20`

TOGGLE · tersine çevir

```
x ^= 1u << 1;
```

	7	6	5	4	3	2	1	0
x	1	0	1	0	0	0	1	0
m	0	0	0	0	0	0	1	0
sonuç	1	0	1	0	0	0	0	0

Bit 1: 1 → 0 (bir daha ^= ile 0 → 1).

`x = 0xA2, m = 1u << 1 = 0x02`

TEST · bit 7 açık mı?

```
(x & (1u << 7)) != 0
```

	7	6	5	4	3	2	1	0
x	1	0	1	0	0	0	1	0
m	1	0	0	0	0	0	0	0
sonuç	1	0	0	0	0	0	0	0

Sonuç ≠ 0 → bit 7 açık → ifade 1

`x = 0xA2, m = 1u << 7 = 0x80`

- Maske: `m = 1u << k` (yalnızca k'inci bit 1).
- **Set** `x |= m`
- **Clear** `x &= ~m`
- **Toggle** `x ^= m`
- **Test** `(x & m) != 0` ← parantez şart!

Kaydırma: 2'nin kuvvetleriyle çarpma ve bölme

unsigned int x = 5; (8 bitlik görünüm)

	7	6	5	4	3	2	1	0	
x	0	0	0	0	0	1	0	1	= 5
x << 1	0	0	0	0	1	0	1	0	= 10 (x · 2)
x << 3	0	0	1	0	1	0	0	0	= 40 (x · 8)
x >> 1	0	0	0	0	0	0	1	0	= 2 (x / 2, kesir atılır)

↑ sağdaki 1 düştü

Sola kaydırmada sağdan, işaretsiz sağa kaydırmada soldan 0 girer.

İşaretli negatif sayı: int n = -17; (8 bitlik görünüm, ikiye tümleyen)

n	1	1	1	0	1	1	1	1	= -17
n >> 2	1	1	1	1	1	0	1	1	= -5 (gcc: soldan işaret biti girer)

- İşaretsizde `x << k` = $x \cdot 2^k$, `x >> k` = $x / 2^k$.
- Kaydırma ≥ bit genişliği (`x << 32`): **tanımsız**.
- Negatif değeri **sola** kaydırmak: **tanımsız**.
- Negatifi **sağa**: **uygulama tanımlı** (gcc: aritmetik).
- `-17 / 4` = −4 ama `-17 >> 2` = **−5**.

Kaputun altında: derleyici x * 8'i nasıl yapıyor?

H04_K10_kaydirma_asm.c · satır 8–21

```c
8  unsigned int carp8(unsigned int x)
9  {
10     return x * 8;
11 }
12
13 unsigned int bol8(unsigned int x)
14 {
15     return x / 8;
16 }
17
18 int bol8_isaretli(int x)
19 {
20     return x / 8;
21 }
```

GCC -O2 -S (KESİT)

```
carp8:
  endbr64
  leal 0(,%rdi,8), %eax
  ret
bol8:
  endbr64
  movl %edi, %eax
  shrl $3, %eax
  ret
bol8_isaretli:
  endbr64
  testl %edi, %edi
  leal 7(%rdi), %eax
  cmovns %edi, %eax
  sarl $3, %eax
  ret
```

`x * 8` → `leal` (adres hesaplama devresi): **çarpma komutu yok**.

unsigned x / 8 → `shrl $3`.

int x / 8 → önce **+7** düzeltmesi, sonra `sarl $3`: `/` sıfıra doğru, `>>` aşağı yuvarlar.

Sonuç: `x << 3` yazmak hızlandırmaz; **niyetinizi açık yazın**.

Örnek 9 ★★ Tek mi, çift mi? % 2 ve & 1

H04_09_tek_cift.c · satır 15–20

```
15      printf("n %% 2          = %d\n", n % 2);
16      printf("n & 1          = %d\n", n & 1);
17      printf("n %% 2 != 0     = %d   (tek mi?)\n", n % 2 != 0);
18      printf("(n & 1) == 1   = %d   (tek mi?)\n", (n & 1) == 1);
19      printf("n %% 2 == 1     = %d   (dikkat: negatif tek sayıda 0)\n",
20             n % 2 == 1);
```

GERÇEK ÇIKTI · GİRDİ -7

```
$ ./tek_cift      # negatif tek sayı
Bir tamsayı: -7
n % 2          = -1
n & 1          = 1
n % 2 != 0     = 1   (tek mi?)
(n & 1) == 1   = 1   (tek mi?)
n % 2 == 1     = 0   (dikkat: negatif tek sayıda 0)
```

Ağırlıklar: 128 64 32 16 8 4 2 1 — yalnızca en sağdaki ($2^0 = 1$) tektir

	7	6	5	4	3	2	1	0	
14	0	0	0	0	1	1	1	0	n & 1 = 0 → çift
7	0	0	0	0	0	1	1	1	n & 1 = 1 → tek
-7	1	1	1	1	1	0	0	1	n & 1 = 1 → tek (-7 % 2 = -1!)

`-7 % 2` = **−1** → `n % 2 == 1` negatifte **yanlış**!
Taşınabilir test: `n % 2 != 0`; `n & 1` LSB'yi okur.

03

Öncelik, yan etkiler ve tip dönüşümleri

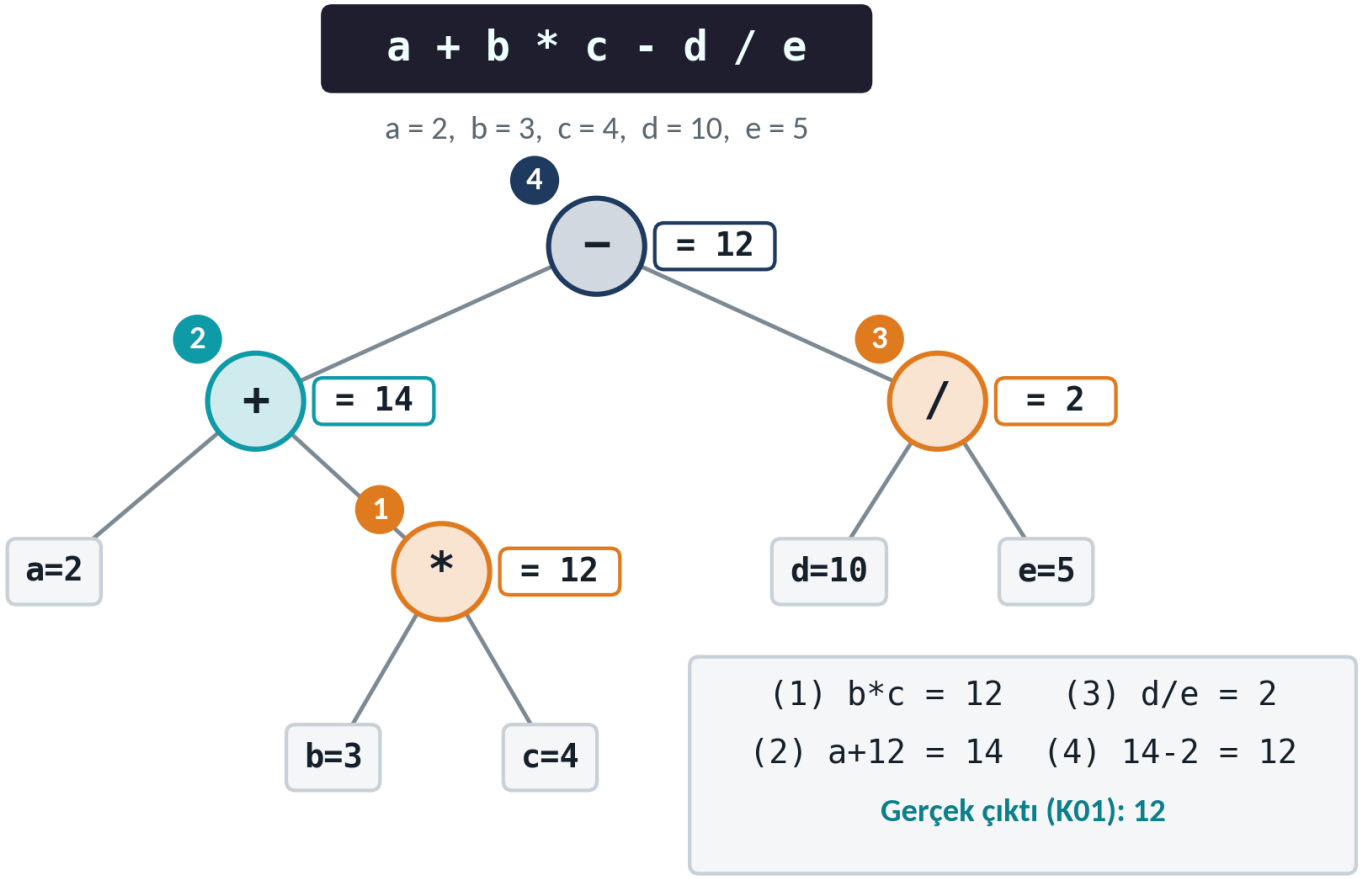
İfade ağacı · tanımsız davranış · yükseltme · cast · genişletme/daraltma · math.h

Operatör önceliği ve birleşme yönü

Düzey	Operatörler	Anlamı	Yön
1	`() [] -> . x++ x--`	sonek, çağrı, dizi, üye	soldan sağa
2	`++x --x +x -x ! ~ (tip) sizeof *p &x`	tekli (unary)	sağdan sola
3	`* / %`	çarpma, bölme, kalan	soldan sağa
4	`+ -`	toplama, çıkarma	soldan sağa
5	`<< >>`	kaydırma	soldan sağa
6	`< <= > >=`	ilişkisel	soldan sağa
7	`== !=`	eşitlik	soldan sağa
8	`&`	bit düzeyi VE	soldan sağa
9	`^`	bit düzeyi ÖZEL VEYA	soldan sağa
10	`|`	bit düzeyi VEYA	soldan sağa
11	`&&`	mantıksal VE	soldan sağa
12	`||`	mantıksal VEYA	soldan sağa
13	`?:`	koşul (H5)	sağdan sola
14	`= += -= *= /= %= &= |= ^= <<= >>=`	atama	sağdan sola
15	`,`	virgül	soldan sağa

- Üstteki satır **daha sıkı** bağlar.
- Aritmetik **soldan sağa**: `10 - 4 - 3` = 3.
- Tekli ve atama **sağdan sola**: `x = y = z = 7`.
- `==` ve `!=`, `&`'ten sıkı: `x & m != 0` ✗
- `+`, `<<`'den sıkı: `1 << n + 1` = `1 << (n+1)`
- Şüphedeysen **parantez**.

İfade ağacı: a + b * c - d / e



- Yapraklar **işlenen**, iç düğümler **operatör**.
- En son uygulanan operatör **kökte**: `-`.
- `(a + (b * c)) - (d / e)` = 12 (K01 çıktısı).
- Öncelik **gruplamayı** belirler; `b*c` mi `d/e` mi önce? **Belirtilmemiş**.

TAHMİN ET

Tahmin et: bu satır ne yazdırır?

H04_Q03_oncelik_mantik.c · satır 8–8

```
8       printf("%d\n", 2 + 3 * 4 > 10 && 5 % 3 == 2);
```

A 0

B 1

C 14

D Derlenmez

Önce kâğıda yazın, eşinizle karşılaştırın; sonra programı çalıştırıp gerçek çıktıyla doğrulayın.

Yan etkiler ve tanımsız davranış: i = i++ + ++i

hata_avcisi/H04_HA5_sirasiz_yan_etki.c · satır 10–14

```c
10      i = i++ + ++i;          /* HATA: i aynı ifadede birden çok kez değişir */
11      printf("i = %d\n", i);
12
13      int n = 5;
14      int k = ++n + ++n;      /* HATA: n iki kez değişir; sıra belirsiz */
```

GCC 13 (GERÇEK)

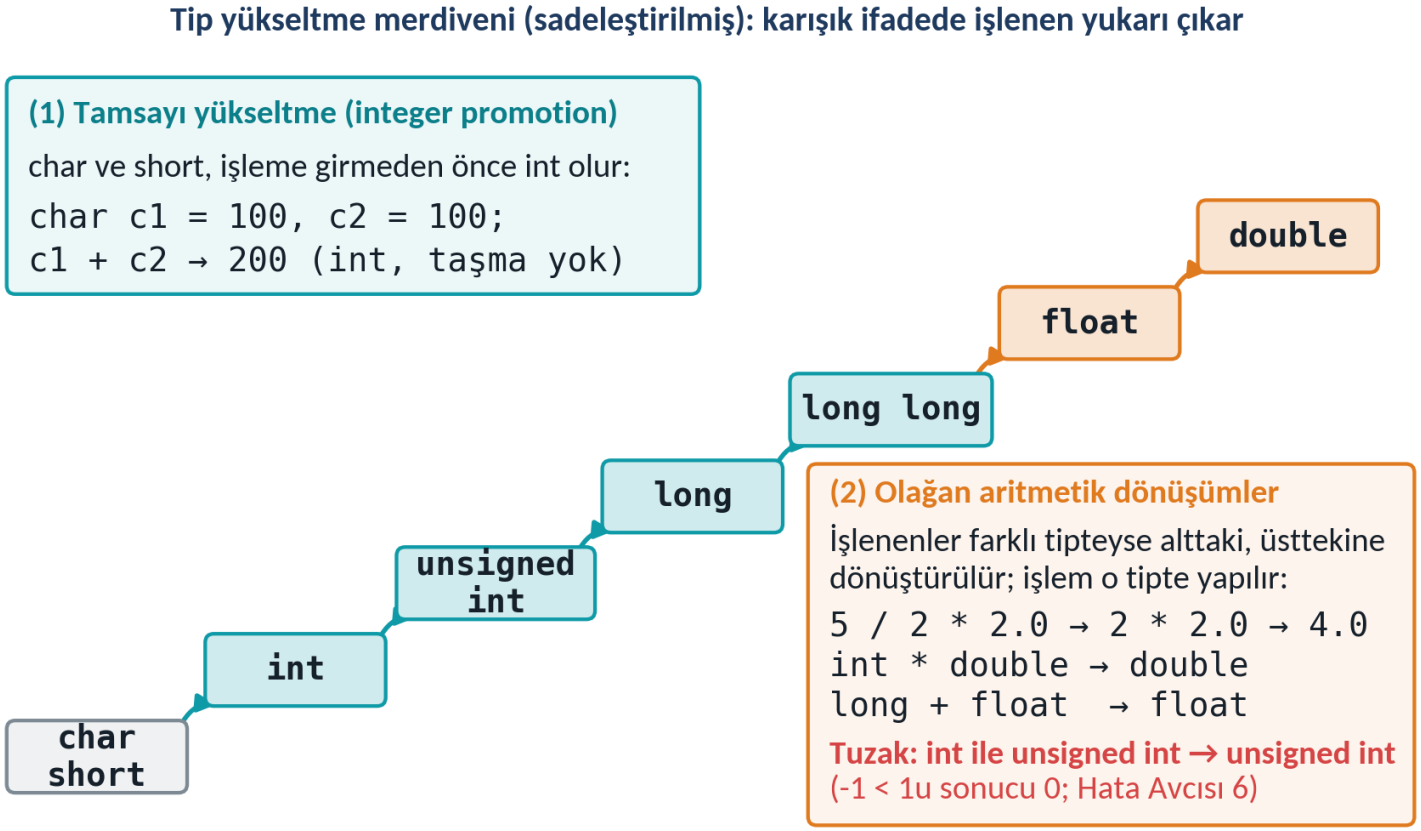
```
10:7: warning: operation on ‘i’ may be undefined
      [-Wsequence-point]
$ ./ha5
i = 12
k = 14
```

CLANG 18 (GERÇEK)

```
10:10: warning: multiple unsequenced modifications to 'i'
      [-Wunsequenced]
$ ./ha5_x
i = 12
k = 13
```

Aynı kod: k GCC’de **14**, Clang’da **13**. İkisi de standarda uygun: aynı nesneyi **sıralanmamış** iki yan etkiyle değiştirmek **tanımsızdır**. Soru “ne yazdırır?” değil, **“neden yazılmamalı?”**

Örtük dönüşümler: tip yükseltme merdiveni



- `char`, `short` → işlemden önce **int** (yükseltme).
- İki işlenen farklı tipteyse alttaki **üsttekine** dönüşür.
- `sizeof('A')` = **4**: C'de karakter sabiti `int`.
- `unsigned char u = 255; u = u + 1;` → **0**.
- **int + unsigned** → unsigned: `-1 < 1u` → **0**!

Açık dönüşüm (cast): nereye yazdığınız önemli

H04_06_bolme_donusum.c · satır 10–14

```c
10      printf("7 / 2           = %d\n", 7 / 2);
11      printf("7.0 / 2         = %f\n", 7.0 / 2);
12      printf("7 / 2.0         = %f\n", 7 / 2.0);
13      printf("(double)7 / 2   = %f\n", (double)7 / 2);
14      printf("(double)(7 / 2) = %f\n", (double)(7 / 2));
```

GERÇEK ÇIKTI (İLK 5 SATIR)

```
$ ./bolme
7 / 2           = 3
7.0 / 2         = 3.500000
7 / 2.0         = 3.500000
(double)7 / 2   = 3.500000
(double)(7 / 2) = 3.000000
```

- Cast **tekli** operatördür, `/`'den sıkı bağlar.
- `(double)7 / 2` → 7.0 / 2 = **3.5**.
- `(double)(7 / 2)` → önce `7 / 2` = 3 → **3.0**.
- Ortalama kalıbı: `(double)toplam / adet`.

Hata Avcısı 6: -1 < 1u neden yanlış?

Hatalı: -5 < 10 → 0

hata_avcisi/H04_HA6_isaret_karsilastirma.c · satır 9–14

```c
 9      int sicaklik = -5;
10      unsigned int esik = 10;
11
12      int soguk_mu = sicaklik < esik;   /* HATA: -5 işaretsize dönüşür */
13      printf("-5 < 10 mu? %d\n", soguk_mu);
14      printf("-1 < 1u mu? %d\n", -1 < 1u);
```

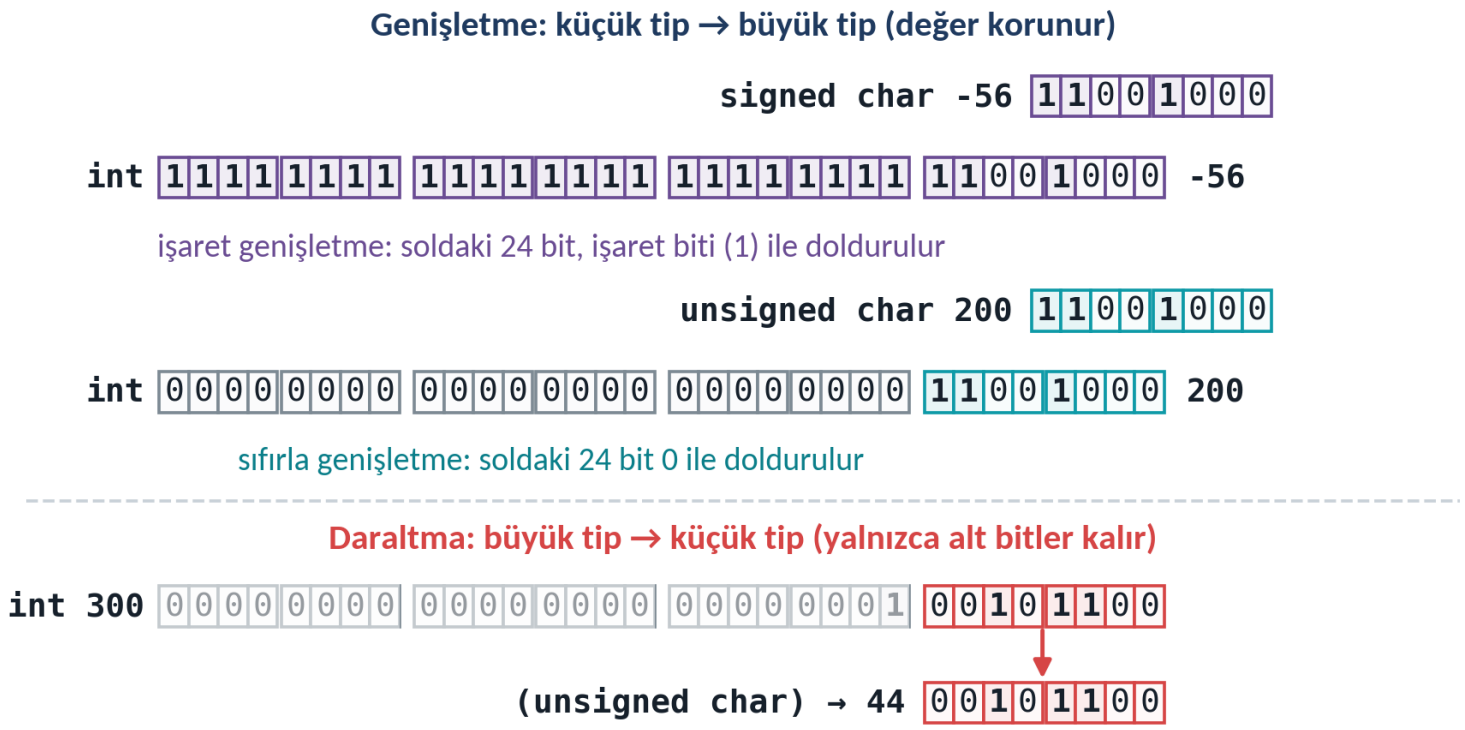
Düzeltilmiş: aynı işaretli tip

hata_avcisi/H04_HA6_isaret_karsilastirma_duzeltilmis.c · satır 9–14

```c
 9      int sicaklik = -5;
10      int esik = 10;                  /* iki işlenen de işaretli */
11
12      int soguk_mu = sicaklik < esik;
13      printf("-5 < 10 mu? %d\n", soguk_mu);
14      printf("-1 < 1 mi?  %d\n", -1 < 1);
```

`int` ile `unsigned int` → **unsigned**: −5 = 4 294 967 291. GCC yalnızca `-Wextra` ile uyardı: `comparison of integer expressions of different signedness [-Wsign-compare]`.

Kaputun altında: genişletme ve daraltmada bitler



- **İşaret genişletme:** üst bitler işaret bitiyle dolar (−56 → `0xFFFFFFC8`).
- **Sıfırla genişletme:** işaretsizde 0 ile (200 → `0x000000C8`).
- **Daraltma:** alt bitler kalır: `(unsigned char)300` = 44.
- İşaretli hedefe sığmıyorsa: **uygulama tanımlı** (gcc: `(short)70000` = 4464).
- float → int sığmıyorsa: **tanımsız** (Ariane).

math.h ve -lm: bağlayıcı hatasını tanımak

Fonksiyon	Gerçek sonuç (K08)
`sqrt(2.0)`	`1.414214`
`pow(2.0, 10)`	`1024.0`
`fabs(-2.7)`	`2.7`
`floor(-2.7)`	`-3.0`
`ceil(-2.7)`	`-2.0`
`round(2.5)`	`3.0`
`round(-2.5)`	`-3.0`

- Hepsi `double` alır ve döndürür.
- Linux'ta matematik kitaplığı **ayrıdır**: `-lm` ekleyin.
- **Bağlayıcı** hatası: bildirim `<math.h>`'te var, **gövde** yok (H1).
- Sabit argümanları (`sqrt(2.0)`) gcc **derlemede** hesaplar: K08'de yalnızca `round(x)` hata verdi.
- Windows/MinGW ve macOS'ta genellikle gerekmez; eklemek zararsız.

GERÇEK BAĞLAYICI HATASI: ÖRNEK 11, -LM UNUTULDU

```
$ gcc -std=c17 … H04_11_ikinci_derece.c -o kokler
ld: /tmp/ccXXXXXX.o: in function `main':
H04_11_ikinci_derece.c:16:(.text+0x83): undefined reference to `pow'
ld: H04_11_ikinci_derece.c:17:(.text+0xbe): undefined reference to `sqrt'
ld: H04_11_ikinci_derece.c:18:(.text+0x10f): undefined reference to `sqrt'
collect2: error: ld returned 1 exit status
```

Yuvarlama: dört yol, dört sonuç

x	(int)x	floor(x)	ceil(x)	round(x)	(int)(x+0.5)	%.0f
2.5	2	2	3	3	3	**2**
-2.5	-2	-3	-2	-3	-2	-2
2.7	2	2	3	3	3	3
-2.7	-2	-3	-2	-3	**-2**	-3

trunc: sıfıra doğru · floor: aşağı · ceil: yukarı · round: en yakına, yarımda sıfırdan uzağa

printf %.0f (glibc): yarımda ÇİFTE yuvarlar (2.5 → 2) · (int)(x + 0.5) negatiflerde yanlış (−2.7 → −2)

Kırmızı hücreler beklenmedik sonuçlar. Değerler H04_K14_yuvarlama_tablosu.c programının gerçek çıktısıdır.

- `(int)x` sıfıra doğru keser.
- `round` yarımda **sıfırdan uzağa**: 2.5 → 3.
- `printf("%.0f")` glibc'de yarımda **çifte**: 2.5 → "2".
- `(int)(x + 0.5)` negatifte **yanlış**: −2.7 → −2.
- Not ≥ 0 olduğu için Mini-OBS'de `+0.5` kalıbı geçerli.

04

Örnek merdiveni

13 örnek: dört işlemden dosya izinlerine

Örnek 2 ve 3: / ve % birlikte

Örnek 2 ★ · saniye → sa:dk:sn

H04_02_saniye_donustur.c · satır 16–19

```
16      int saat = toplam_saniye / 3600;
17      int kalan = toplam_saniye % 3600;
18      int dakika = kalan / 60;
19      int saniye = kalan % 60;
```

GERÇEK ÇIKTI · GİRDİ 3725

```
3725 saniye = 01:02:05
```

`3725 / 3600 = 1`	saat
`3725 % 3600 = 125`	kalan
`125 / 60 = 2`	dakika
`125 % 60 = 5`	saniye

→ **01:02:05**

Örnek 3 ★★ · basamaklar

H04_03_basamaklar.c · satır 16–18

```
16      int birler = sayi % 10;
17      int onlar = (sayi / 10) % 10;
18      int yuzler = sayi / 100;
```

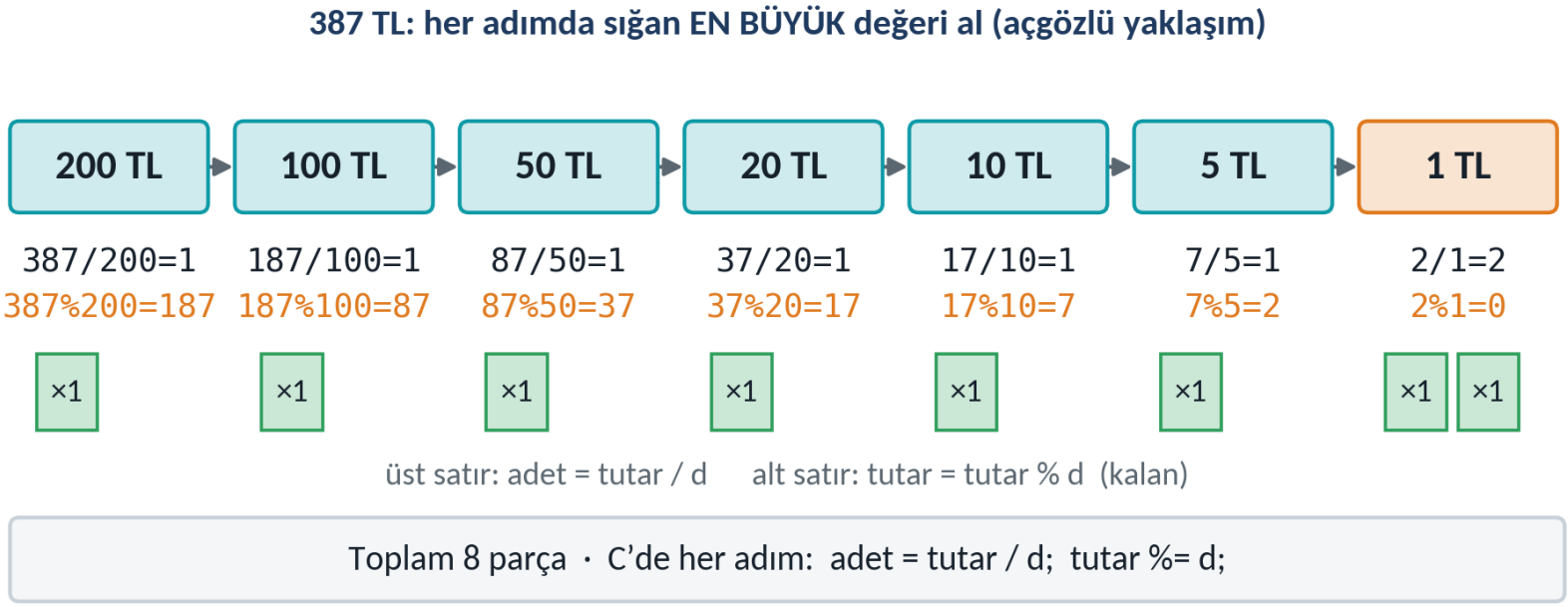
GERÇEK ÇIKTI · GİRDİ 905

```
Yüzler: 9  Onlar: 0  Birler: 5
```

`905 % 10 = 5`	birler
`905 / 10 = 90`	son basamak atıldı
`90 % 10 = 0`	onlar
`905 / 100 = 9`	yüzler

9 0 5

Örnek 4 ★★ (amiral) Para üstü: açgözlü yaklaşım



- **Girdi:** tutar (TL, tamsayı ≥ 0).
- **İşlem:** büyükten küçüğe her d için `adet = tutar / d; tutar %= d;`
- **Çıktı:** her değerden adet, toplam parça.
- **Açgözlü:** her adımda sığan **en büyük** değer.
- {1, 3, 4} ile 6: açgözlü 4+1+1, en iyisi 3+3 (AP-2).

Örnek 4: ardışık bölme ve kalan (%=)

H04_04_para_ustu.c · satır 16–28

```c
    int adet200 = tutar / 200;   /* önce en büyük değer */
    tutar %= 200;                /* tutar = tutar % 200 */
    int adet100 = tutar / 100;
    tutar %= 100;
    int adet50 = tutar / 50;
    tutar %= 50;
    int adet20 = tutar / 20;
    tutar %= 20;
    int adet10 = tutar / 10;
    tutar %= 10;
    int adet5 = tutar / 5;
    tutar %= 5;
    int adet1 = tutar;           /* kalan: 1 TL'ler */
```

GERÇEK ÇIKTI · GİRDİ 387

```
$ ./para_ustu
Para üstü (TL, tamsayı): 387
387 TL için en az sayıda banknot/madeni para:
200 TL x 1
100 TL x 1
 50 TL x 1
 20 TL x 1
 10 TL x 1
  5 TL x 1
  1 TL x 2
Toplam parça: 8
```

- Her değer için **iki satır**: / adet, %= kalan.
- 387 → 187 → 87 → 37 → 17 → 7 → 2: **8 parça**.

Örnek 7 ★★ Not ortalamasında cast ve yuvarlama

H04_07_not_ortalamasi.c · satır 18–25

```c
18      double ortalama = (double)toplam / 4;     /* cast: kesirli bölme */
19
20      printf("Toplam               : %d\n", toplam);
21      printf("toplam / 4           : %d\n", tamsayi_ort);
22      printf("(double)toplam / 4   : %.2f\n", ortalama);
23      printf("round(ortalama)      : %.0f\n", round(ortalama));
24      printf("(int)(ortalama + 0.5): %d\n", (int)(ortalama + 0.5));
25      printf("printf ile %%.0f      : %.0f\n", ortalama);
```

ÇIKTI

```
$ ./not_ort
Dört quiz notu: 70 71 70 71
Toplam               : 282
toplam / 4           : 70
(double)toplam / 4   : 70.50
round(ortalama)      : 71
(int)(ortalama + 0.5): 71
printf ile %.0f      : 70
```

Örnek 8 ★★ Artık yıl: tek bir mantıksal ifade

H04_08_artik_yil.c · satır 15–21

```c
15      int artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
16
17      printf("%d %% 4   == 0 : %d\n", yil, yil % 4 == 0);
18      printf("%d %% 100 != 0 : %d\n", yil, yil % 100 != 0);
19      printf("%d %% 400 == 0 : %d\n", yil, yil % 400 == 0);
20      printf("Artık yıl mı? (1 evet, 0 hayır): %d\n", artik);
21      printf("Şubat bu yıl %d gün çeker.\n", 28 + artik);
```

ÇIKTI

```
$ ./artik_yil
Yıl: 1900
1900 % 4   == 0 : 1
1900 % 100 != 0 : 0
1900 % 400 == 0 : 0
Artık yıl mı? (1 evet, 0 hayır): 0
Şubat bu yıl 28 gün çeker.
```

Örnek 11 ★★ İkinci dereceden denklem: sqrt, pow, -lm

H04_11_ikinci_derece.c · satır 16–18

```c
16      double delta = pow(b, 2) - 4 * a * c;          /* Δ = b² - 4ac */
17      double kok1 = (-b + sqrt(delta)) / (2 * a);
18      double kok2 = (-b - sqrt(delta)) / (2 * a);
```

GERÇEK ÇIKTI · İKİ GERÇEL KÖK

```
$ ./kokler
a, b, c katsayıları: 1 -3 2
Δ  = 1.00
x1 = 2.0000
x2 = 1.0000
```

GERÇEK ÇIKTI · Δ < 0

```
$ ./kokler
a, b, c katsayıları: 1 2 5
Δ  = -16.00
x1 = -nan
x2 = -nan
```

`(2 * a)` parantezi şart: `/ 2 * a` önce böler, sonra **çarpar**.

Δ < 0 → `sqrt` **NaN** döndürür, glibc `-nan` yazar: çökme yok, anlam yok.

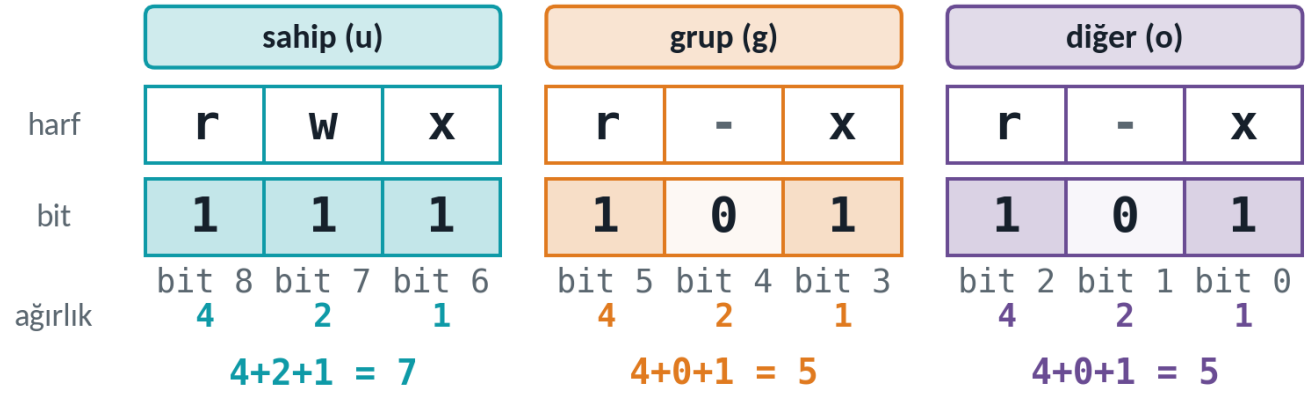
H5 köprüsü: `if (delta < 0)` → "gerçel kök yok".

Örnek 12 ★★★ (amiral) Dosya izin bitleri: rwx

```
$ stat -c '%A %a' rapor.txt
```

```
-rwxr-xr-x 755
```

gerçek çıktı (chmod 755 sonrası)



111 101 101 (ikili) = 7 5 5 (sekizli) = 0755 (C) = 493 (onlu)

Her sekizlik basamak tam olarak 3 biti kodlar: r = 4, w = 2, x = 1.

Dikkat: C'de baştaki 0 sekizlik taban demektir. 755 ile 0755 farklı sayılardır!

- 9 bit: **sahip · grup · diğer** × r w x.
- r = 4, w = 2, x = 1 → her sekizlik basamak **3 bit**.
- `chmod 755` → `rwxr-xr-x` (gerçek `stat` çıktısı).
- C'de `0755` sekizliktir (= 493); `755` ise onluk, **başka** bir sayı!
- Maskeler: `0400` (sahip okuma) … `0001` (diğer çalıştırma).

Örnek 12: set, clear, toggle, test

H04_12_dosya_izinleri.c · satır 15–24

```c
15      unsigned int izin = 0755;              /* baştaki 0: sekizlik taban! */
16      printf("Başlangıç             : %03o\n", izin);
17      printf("Grup yazabilir mi?    : %d\n", (izin & GRUP_W) != 0);  /* test */
18
19      izin |= GRUP_W;                        /* set: biti 1 yap */
20      printf("izin |= GRUP_W        : %03o\n", izin);
21      izin &= ~(DIGER_R | DIGER_X);          /* clear: bitleri 0 yap */
22      printf("izin &= ~(R|X diğer) : %03o\n", izin);
23      izin ^= SAHIP_X;                       /* toggle: tersine çevir */
24      printf("izin ^= SAHIP_X       : %03o\n", izin);
```

GERÇEK ÇIKTI (İLK 6 SATIR)

```
Başlangıç             : 755
Grup yazabilir mi?    : 0
izin |= GRUP_W        : 775
izin &= ~(R|X diğer) : 770
izin ^= SAHIP_X       : 670
izin ^= SAHIP_X       : 770
```

- **Test:** `(izin & GRUP_W) != 0` → 0755'te 0: grup yazamaz.
- **Set:** `|=` → 775. **Clear:** `&= ~(…)` → 770.
- **Toggle:** `^=` iki kez: 670 → 770, eski hâline döner.

MERAK KUTUSU

Örnek 13 ★★★ Merak Kutusu: XOR ile takas

H04_13_xor_takas.c · satır 13–16

```
13      a ^= b;                         /* a = a ^ b */
14      b ^= a;                         /* b = b ^ (a ^ b) = eski a */
15      a ^= b;                         /* a = (a ^ b) ^ eski a = eski b */
16      printf("XOR takası : a = %d, b = %d\n", a, b);
```

GERÇEK ÇIKTI

```
$ ./xor_takas
Başta      : a = 12, b = 25
XOR takası : a = 25, b = 12
Geri takas : a = 12, b = 25
c kendisiyle: c = 0 (7 kalmalıydı)
```

Adım	a (ikili)	a	b (ikili)	b
Başta	01100	12	11001	25
a ^= b;	10101	21	11001	25
b ^= a;	10101	21	01100	12
a ^= b;	11001	25	01100	12

Aynı nesnede çalışmaz: `c ^= c` → **0** (H12: `takas(&x, &x)`)

Hızlı değildir: derleyici geçici değişkenli takası zaten yazmaçlarda yapar.

Okunmaz: açık kod > zekice kod. XOR'un yeri: eşlik, basit şifreleme.

05

Hata Avcısı ve kapanış

Yedi hata · Mühendis Gözüyle · Mini-OBS v0.4 · AP-2 · çıkış bileti

Hata Avcısı: yedi hata, gerçek iletiler

Hata	Belirti (gerçek)	Çözüm
HA1 `a + b / 2`	80 yerine 115 (uyarı yok)	`(a + b) / 2.0`
HA2 `== yerine =`	puan 100 oldu (uyarı yok)	`(puan == 100)`
HA3 `&& yerine &`	2 & 1 = 0; | sağ tarafı çalıştırdı: 27	`&& ve ||`
HA4 `pow → int, -lm`	undefined reference to `pow' · 1157	`-lm; double tutar`
HA5 `i = i++ + ++i`	[-Wsequence-point]; gcc 14 / clang 13	`Ayrı deyimler`
HA6 `-1 < 1u`	0; [-Wsign-compare] yalnızca -Wextra	`Aynı işaretli tip`
HA7 `(short)40000.0`	−25536 / 32767 / çöp (uyarı yok)	`Aralık denetimi, doyurma`

HA7: Ariane 5'in dönüşümü, C'de

hata_avcisi/H04_HA7_ariane_donusum.c · satır 9–11

```
 9      double yatay_hiz = 40000.0;       /* varsayımsal değer: 16 bite sığmaz */
10
11      short bh = (short)yatay_hiz;      /* HATA: aralık dışı dönüşüm */
```

AYNI KOD, DÖRT DERLEME (GERÇEK; UZUN SATIR KAYDIRILDI)

```
$ gcc … -o ha7   (-O0)
double 40000.0 -> short -25536
$ gcc … -fsanitize=float-cast-overflow -o ha7_x
11:5: runtime error: 40000 is outside the range of
      representable values of type 'short int'
double 40000.0 -> short -25536
$ gcc -O2 -o ha7_x
double 40000.0 -> short 32767
$ clang -O2 -o ha7_x
double 40000.0 -> short 312205000
      ↑ (çalıştırmadan çalıştırmaya değişir)
```

gcc -O0'da bitlere ne oldu?

```
40000.0 → int   0x00009C40
alt 16 bit      0x9C40
ikili  1001 1100 0100 0000
işaret biti 1 → short -25536
```

Tanımsız davranış: uyarı yok, program durmuyor, sonuç derleyiciye göre değişiyor.
Düzeltme: `yatay_hiz >= SHRT_MIN && yatay_hiz <= SHRT_MAX` (0/1) ve `fmin`/`fmax` ile **doyurma** → 32767.

Mühendis gözüyle: bu haftadan dört kural

Daraltan dönüşümü sorgula

`double` → `int`, `int` → `short`: değer **her zaman** sığıyor mu? Sığmazsa ne olmalı: hata mı, doyurma mı? (CERT FLP34-C, INT31-C)

Uyarıları aç, cast ile susturma

`-Wall -Wextra` (HA6), `-Wconversion` (HA4), testte `-fsanitize=undefined,float-cast-overflow` (HA7). Cast derleyiciyi **susturur**.

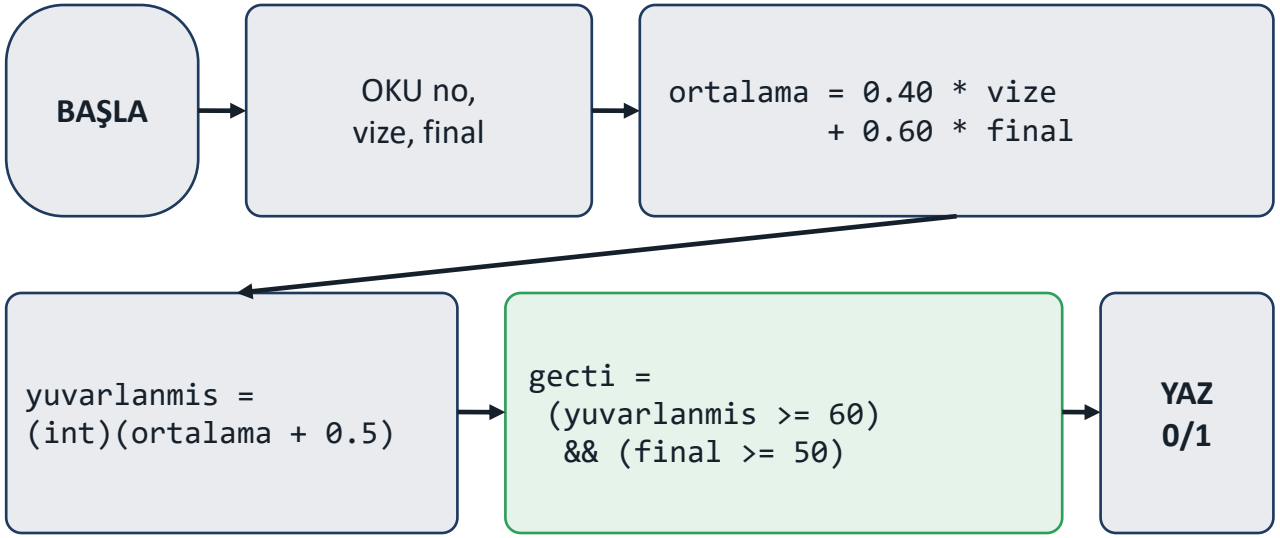
Parantez ve tek yan etki

Öncelik tablosunu okurdan beklemeyin; bir ifadede bir değişkeni **en çok bir kez** değiştirin (CERT EXP30-C).

Mikro-optimizasyon yapma

Derleyici `x * 8`'i `lea`'ya çeviriyor; `x << 3` ya da XOR takası yerine **niyetini açık** yaz. `(lo + hi) / 2` taşabilir: `lo + (hi - lo) / 2`.

Mini-OBS v0.4: ağırlıklı ortalama ve 0/1 geçme durumu



Test	Vize/final	Ort.	Yuv.	Geçti
T1	70 / 85	79.00	79	1
T2	100 / 45	67.00	67	0
T3	73 / 50.5	59.50	60	1
T4	72.5 / 88	81.80	82	1
T5	57 / 61	59.40	59	0
T6	95 / 50	68.00	68	1
T7	**1 / 98.5**	**59.50**	**59**	**0**

H04_K16_miniobs_t7.c · satır 17–19

```
17      double ortalama = VIZE_AGIRLIK * vize + FINAL_AGIRLIK * final;
18      int yuvarlanmis = (int)(ortalama + 0.5);   /* not >= 0: yarım yukarı */
19      int gecti = (yuvarlanmis >= GECME_ORTALAMA) && (final >= FINAL_BARAJI);
```

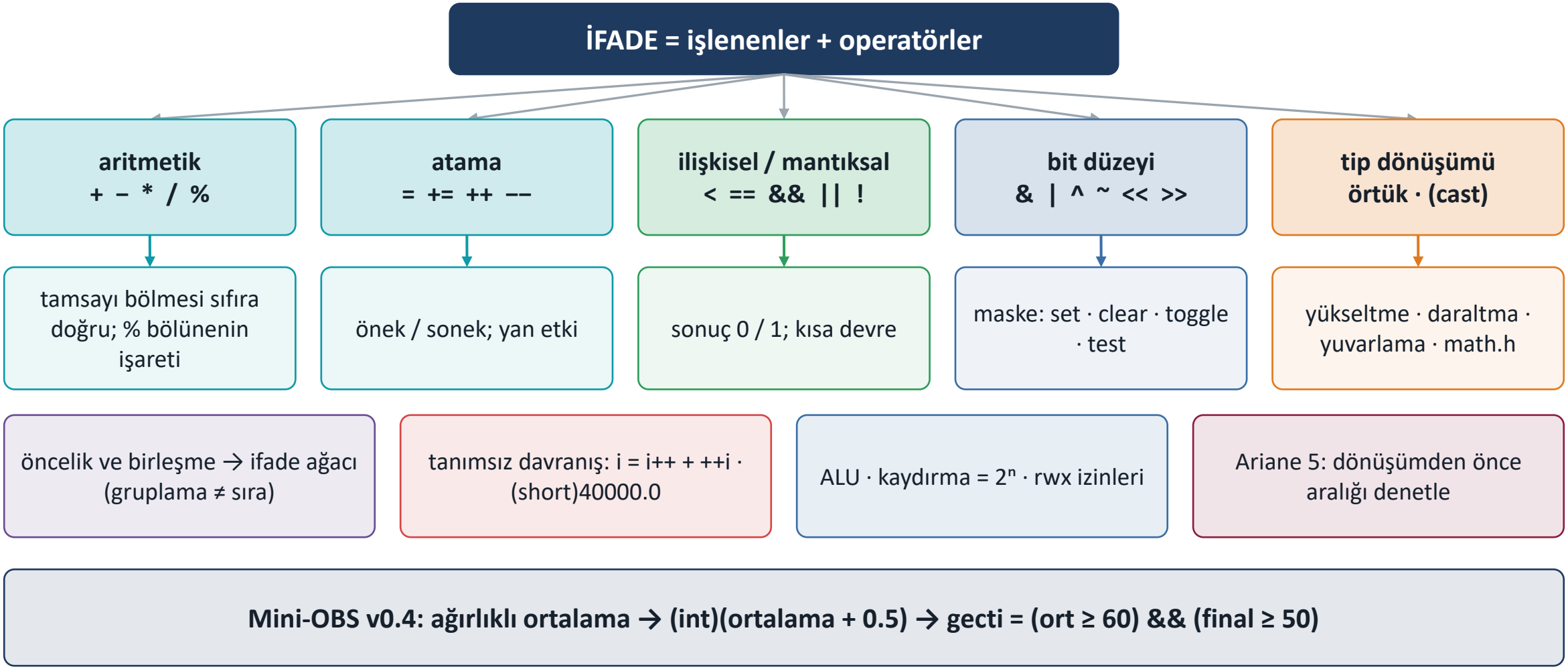
Rapor satırı: `Geçme durumu (1=geçti, 0=kaldı): %d`. **T7:** 1 / 98.5 → matematikte 59.5, hesapta **59.499999999999993** → 59, kaldı (`H04_K16_miniobs_t7.c`).

AP-2 KÖPRÜSÜ

Bu haftanın yapı taşları AP-2'de nerede?

- **Bit maskeleri:** n elemanlı kümenin 2^n alt kümesi $0 \ldots 2^n - 1$ sayılarının bitleridir; `(maske >> k) & 1` → k'inci eleman var mı? (alt küme üretimi)
- **Açgözlü yaklaşım:** para üstü; {1, 3, 4} ile 6'da başarısız → açgözlü seçimin doğruluğunu kanıtlamak.
- **Taşma:** `(dusuk + yuksek) / 2` taşabilir; `dusuk + (yuksek - dusuk) / 2` güvenli (ikili arama).

Özet ve kavram haritası



ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 `printf("%d %d", -7 / 2, -7 % 2);` ne yazdırır? Neden?

2 `unsigned int izin = 0644;` için “grup yazabilir mi?” sorusunu 0/1 veren ifadeyi yazın. `izin & 0020 == 0020` neden yanlış?

3 `double d = 40000.0; short s = (short)d;` satırının C’deki sonucu nedir? Ariane 5 ile ilişkisi?

KAPANIŞ

Haftaya hazırlık ve kaynaklar

- **H5 · Karar yapıları:** `if`, `else`, `switch`; bu haftanın 0/1 ifadeleri koşul olacak.
  - Artık yıl → “Şubat 29 gün”; Δ < 0 → “gerçel kök yok”; b == 0 → bölme yapma.
- **Atölye:** tahmin kartları, 8 LED’lik panel, formülden koda (serbest düşme, bileşik faiz).
- **Mini-OBS v0.4:** kabul ölçütleri ve 7 test ders notunda (Bölüm 7).
- **Web crash course:** bit oynatıcı, ifade ağacı çözümleyici, tip dönüşümü kartları, rwx paneli; 10 soruluk quiz.
- **Okuma:** Kernighan & Ritchie (1988), Bölüm 2; Bryant & O’Hallaron (2016), Bölüm 2.1–2.3; Lions (1996).